



# chapter 10

■ ■ ■ ■ ■

# 内部排序

## 主讲人：周瑜

10.1 概述

10.2 插入排序

10.3 交换排序

10.4 选择排序

10.5 归并排序

10.6 基数排序

## 1. 什么是排序？

将一组杂乱无章的**数据**按一定的**规律**顺次排列起来。

存放在数据表中

按关键字排序

## 2. 排序的目的是什么？

便于查找

### 3. 排序算法的好坏如何衡量？

**时间效率：** 排序速度（即排序所花费的**全部比较次数**）

**空间效率：** 占内存辅助空间的大小

**稳定性：** 若两个记录A和B的关键字值相等，但排序后A、B的先后次序保持不变，则称这种排序算法是稳定的。

## 稳定排序与不稳定排序

假设 $K_i=K_j$ ，且排序前序列中  $R_i$  领先于  $R_j$

➤**稳定排序**：若在排序后的序列中  $R_i$  仍领先于  $R_j$ ，则称排序方法是**稳定的**。

➤**不稳定排序**：若在排序后的序列中 $R_j$ 领先于 $R_i$ ，则称排序方法是**不稳定的**。

## 稳定排序与不稳定排序

例，序列      3    15    8    8    6    9

若排序后得    3    6    8    8    9    15    稳定的

若排序后得    3    6    8    8    9    15    不稳定的

#### 4. 什么叫内部排序？什么叫外部排序？

若待排序记录都在**内存中**，称为内部排序；

若待排序记录一部分在内存，一部分在外存，则称为外部排序。

注：外部排序时，要将数据分批调入内存来排序，中间结果还要及时放入外存，显然外部排序要复杂得多。

## 5. 待排序记录在内存中怎样存储和处理？

- ① 顺序排序：排序时直接移动记录；
- ② 链表排序：排序时只移动指针；
- ③ 地址排序：排序时先移动地址，最后再移动记录。

注：地址排序中可以增设一维数组来专门存放记录的地址。



## 6. 顺序存储（顺序表）的抽象数据类型如何表示？

大多数排序算法都是针对顺序表结构的（便于直接移动元素）

```
# define MAXSIZE 20    // 设记录数不超过20个
typedef int KeyType;    // 设关键字为整型量 (int型)
```

```
typedef struct {        // 定义每个记录（数据元素）的结构
    KeyType    key;      // 关键字
    InfoType    otherinfo; // 其它数据项
}RecordType, node;      // 例如node.key表示其中一个分量
```

```
typedef struct {        // 定义顺序表L的结构
    RecordType r [ MAXSIZE +1 ]; // 存储顺序表的向量，r[0]一般作哨兵或缓冲区
    int length;         // 顺序表的长度
}SqList ,L;            // 例如L.r或L.length表示其中一个分量
```

若r数组是表L中的一个分量且为node类型，  
则r中某元素的key分量表示为：r[i].key

## 7. 内部排序的算法有哪些？

➤ 按排序的规则不同，可分为5类：

插入排序、交换排序、选择排序、归并排序、基数排序

➤ 按排序算法的时间复杂度不同，可分为3类：

简单的排序算法：时间效率低， $O(n^2)$

先进的排序算法：时间效率高， $O(n \log_2 n)$

基数排序算法：时间效率高， $O(d \times n)$

**$d$  = 关键字的位数(长度)**

**10.1 概述**

**10.2 插入排序**

**10.3 交换排序**

**10.4 选择排序**

**10.5 归并排序**

**10.6 基数排序**

插入排序的基本思想：

每步将一个待排序的对象，按其关键码大小，插入到前面已经排好序的一组对象的适当位置上，直到对象全部插入为止。

简言之，边插入边排序，保证子序列中随时都是排好序的。

插入排序有多种具体实现算法：

- 1) 直接插入排序
  - 2) 折半插入排序
  - 3) 2-路插入排序
  - 4) 表插入排序
  - 5) 希尔排序
- 小改进
- 大改进

### 1) 直接插入排序

新元素插入到哪里？

在已形成的有序表中线性查找，并在适当位置插入，把原来位置上的元素向后顺移。

## 1) 直接插入排序

例1：关键字序列 $T=(13, 6, 3, 31, 9, 27, 5, 11)$ ，  
请写出直接插入排序的中间过程序列。

【13】 , 6, 3, 31, 9, 27, 5, 11

【6, 13】 , 3, 31, 9, 27, 5, 11

【3, 6, 13】 , 31, 9, 27, 5, 11

【3, 6, 13, 31】 , 9, 27, 5, 11

【3, 6, 9, 13, 31】 , 27, 5, 11

【3, 6, 9, 13, 27, 31】 , 5, 11

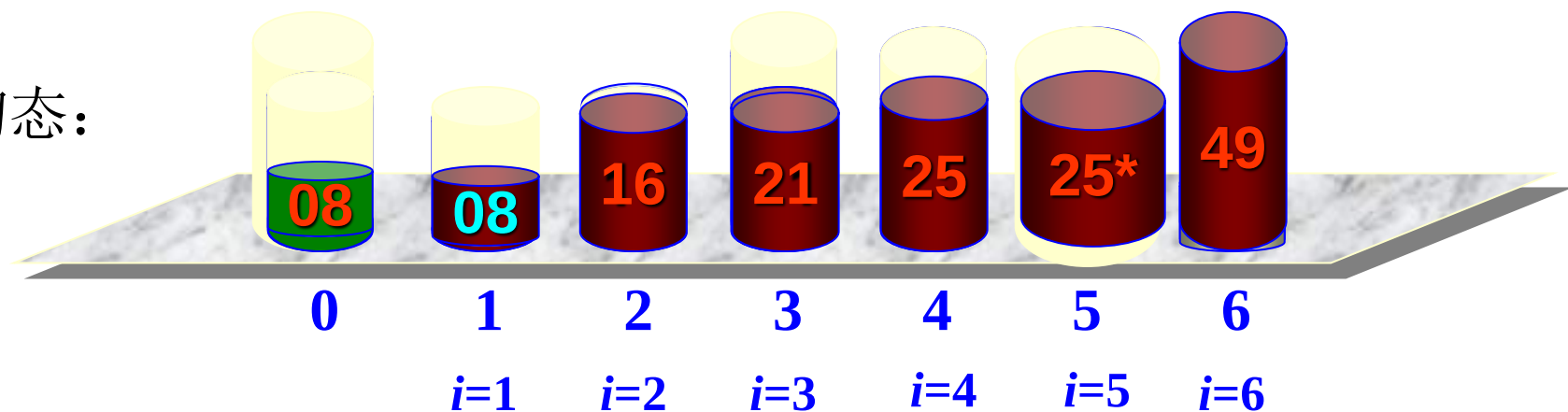
【3, 5, 6, 9, 13, 27, 31】 , 11

【3, 5, 6, 9, 11, 13, 27, 31】

例2：关键字序列 $T = (21, 25, 49, 25^*, 16, 08)$ ，  
请写出直接插入排序的具体实现过程。\*表示后一个25

解：假设该序列已存入一维数组 $r[7]$ 中，将 $r[0]$ 作为哨兵（Temp）。则程序执行过程为：

初态：





**时间效率：**因为在最坏情况下，所有元素的比较次数总和为  $(0+1+\dots+n-1)\rightarrow O(n^2)$ 。其他情况下也要考虑移动元素的次数。故时间复杂度为  $O(n^2)$

**空间效率：**仅占用1个缓冲单元：  $O(1)$

**算法的稳定性：**因为25\*排序后仍然在25的后面：稳定

## 2) 折半插入排序

一个容易想到的改进方法：既然子表有序且为顺序存储结构，则插入时采用折半查找可加速。

折半插入排序基本思想是：

设在顺序表中有一个对象序列  $V[0], V[1], \dots, v[n-1]$ 。其中， $v[0], V[1], \dots, v[i-1]$  是已经排好序的对象。在插入  $v[i]$  时，利用折半搜索法寻找  $v[i]$  的插入位置。

## 2) 折半插入排序

优点：比较次数大大减少，全部元素比较次数仅为  $O(n\log_2 n)$ 。

时间效率：虽然比较次数大大减少，但移动次数并未减少，所以排序效率仍为  $O(n^2)$ 。

空间效率：仍为  $O(1)$

稳定性：稳定

### 3) 2-路插入排序

对折半插入排序的一种改进

目的：减少排序过程中的移动次数

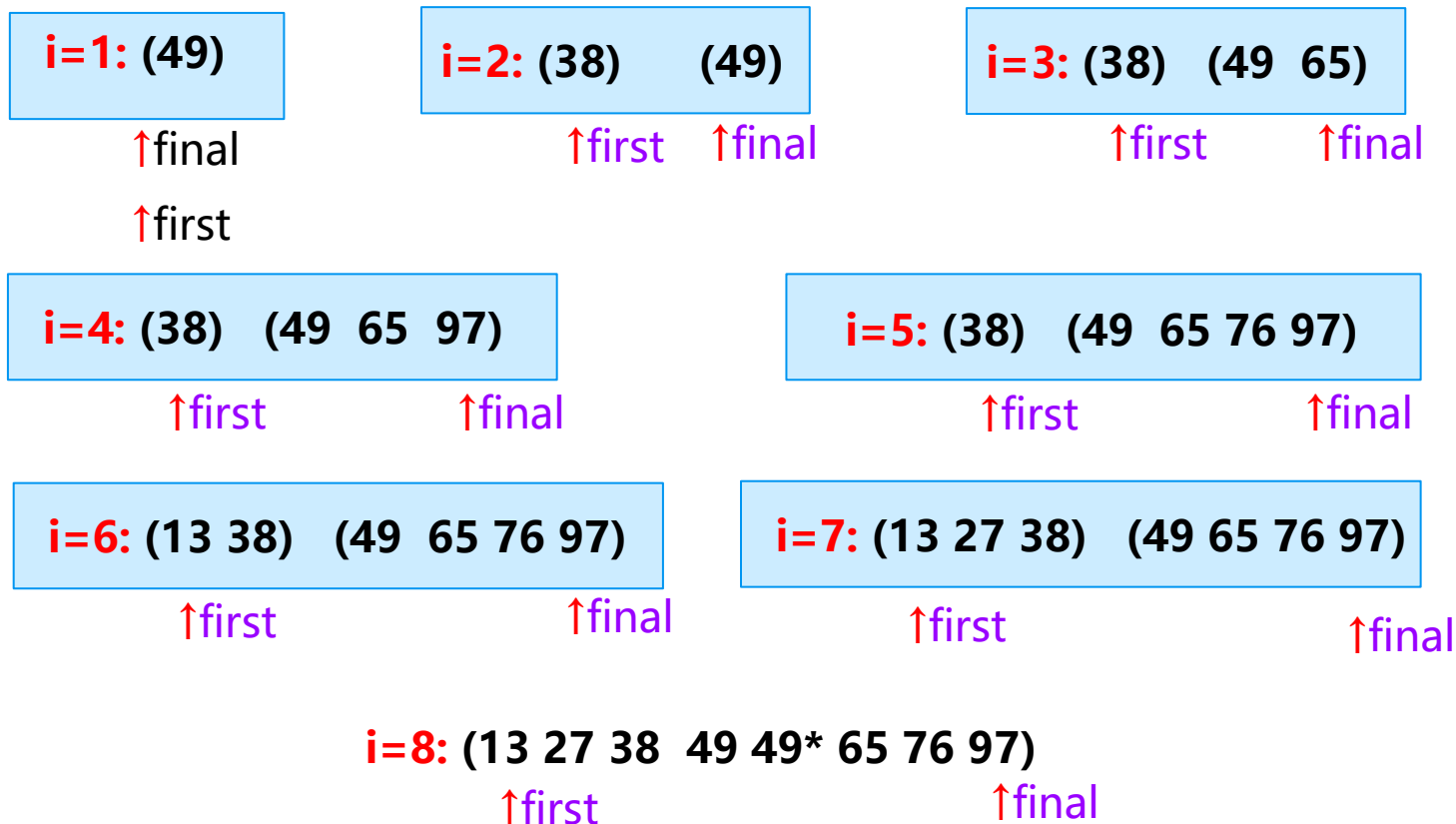
代价：增加辅助空间，增开辅助数组 $d$ , 大小与 $r$ 相同。

## 3) 2-路插入排序

思路：将 $r[1]$ 赋值给 $d[1]$ ；以 $d[1]$ 内容为中值，排序过程中形成两个有序部分；前半部分序列的值都小于 $d[1]$ ，后半部分序列的值都大于 $d[1]$ ；每次将 $r[i]$ 元素逐个与 $d[1]$ 比较， $r[i] < d[1]$ 插入到 $d[1]$ 值之前的有序序列中；  
 $r[i] > d[1]$ 插入到 $d[1]$ 之后的有序序列中。

## 3) 2-路插入排序

例：待排序列T=(49, 38, 65, 97, 76, 13, 27, 49\*)



### 3) 2-路插入排序

在2-路插入排序中，移动记录的次数约为 $n^2/8$ 。因此，2-路插入排序**只能减少移动次数**，而不能绝对避免移动记录。

当 $r[1]$ 是待排序列关键字中最小或最大的记录时，2-路插入排序就完全失去其优越性。

讨论：若记录是链表结构，用直接插入排序行否？

答：行，而且无需移动元素，时间效率更高！

**但请注意：单链表结构无法实现“折半查找”**



#### 4) 表插入排序

基本思想：在顺序存储结构中，给每个记录增开一个指针分量，在排序过程中将指针内容逐个修改为已经整理（排序）过的后继记录地址。

优点：在排序过程中不移动元素，只修改指针。

此方法具有链表排序和地址排序的特点

例：关键字序列 $T=(21, 25, 49, 25^*, 16, 08)$ ，请写出表插入排序的具体实现过程。

\*表示后一个25

解：假设该序列（结构类型）已存入一维数组 $r[7]$ 中，将 $r[0]$ 作为表头结点。则算法执行过程为：

| $i$   | 关键字 $r[i].key$ | 指针 $r[i].link$ |
|-------|----------------|----------------|
| 初态    | <b>MaxNum</b>  | 6              |
| $i=1$ | 21             | 2              |
| $i=2$ | 25             | 4              |
| $i=3$ | 49             | 0              |
| $i=4$ | 25*            | 3              |
| $i=5$ | 16             | 1              |
| $i=6$ | 08             | 5              |

表头结点

指向第1个元素

## 4) 表插入排序

```
int LinkInsertSort (Linklist &L) {  
    L.r[0].Key = MaxNum;  
    L.r[0].Link = 1;  
    L.r[1].Link = 0;           //形成循环链表  
  
    for ( int i = 2; i <= L.length; i++ ) {  
        int current = L.r[0]. Link;    //current=当前记录指针  
        int pre = 0;                   //pre=当前记录current的前驱指针  
        while ( L.r[current]. Key <= L.r[i]. Key) {  
            pre = current;              //current指针准备后移, pre跟上;  
            current = L.r[current]. Link; } //找插入位置(即p=p->link)  
  
        L.r[i]. Link = current; //新记录r[i]找到合适序位开始插入  
        L.r[pre]. Link = i;    //在pre与current之间链入  
    } //for  
} //LinkInsertSort
```

表插入排序算法分析：

- ① 无需移动记录，只需修改 $2n$ 次指针值。但由于比较次数没有减少，故时间效率仍为 $O(n^2)$ 。
- ② 空间效率肯定低，因为增开了指针分量（但在运算过程中没有用到更多的辅助单元）。
- ③ 稳定性：稳定。

表插入排序算法分析：

注：此算法得到的只是一个有序链表，查找记录时只能满足顺序查找方式。

改进：可以根据表中指针线索，很快对所有记录重排，形成真正的有序表（顺序存储方式），从而能满足折半查找方式。

### 5) 希尔 (shell) 排序

又称缩小增量排序

基本思想：

先将整个待排记录序列分割成若干子序列，分别进行直接插入排序，待整个序列中的记录“基本有序”时，再对全体记录进行一次直接插入排序。

### 5) 希尔 (shell) 排序

技巧:

子序列的构成不是简单地“逐段分割”，而是将相隔某个增量 $dk$ 的记录组成一个子序列,让增量 $dk$ 逐趟缩短（例如依次取5,3,1），直到 $dk=1$ 为止。

优点:

让关键字值小的元素能很快前移，且序列若基本有序时，再用直接插入排序处理，时间效率会高很多。

例：关键字序列  $T=(49, 38, 65, 97, 76, 13, 27, 49^*, 55, 04)$ ，  
请写出希尔排序的具体实现过程。

| $r[i]$         | 0 | 1  | 2  | 3   | 4  | 5   | 6  | 7  | 8   | 9  | 10 |
|----------------|---|----|----|-----|----|-----|----|----|-----|----|----|
| 初态:            |   | 49 | 38 | 65  | 97 | 76  | 13 | 27 | 49* | 55 | 04 |
| 第1趟 ( $dk=5$ ) |   | 13 | 27 | 49* | 55 | 04  | 49 | 38 | 65  | 97 | 76 |
| 第2趟 ( $dk=3$ ) |   | 13 | 04 | 49* | 38 | 27  | 49 | 55 | 65  | 97 | 76 |
| 第3趟 ( $dk=1$ ) |   | 04 | 13 | 27  | 38 | 49* | 49 | 55 | 65  | 76 | 97 |

算法分析：开始时  $dk$  的值较大，子序列中的对象较少，排序速度较快；随着排序进展， $dk$  值逐渐变小，子序列中对象个数逐渐变多，由于前面工作的基础，大多数对象已基本有序，所以排序速度仍然很快。



### 5) 希尔 (shell) 排序

时间效率:  $O(n^{1.25}) \sim O(1.6n^{1.25})$  ——由经验公式得到

空间效率:  $O(1)$  , 因为仅占用1个缓冲单元

算法的稳定性: 不稳定, 因为49\*排序后却到了49的前面

例：关键字序列(256, 301, 751, 129, 937, 863, 742, 694, 076, 438)为例，写出执行希尔排序（取 $dk=5, 3, 1$ ）算法的各趟排序结束时，关键字序列的状态。

原始序列：256, 301, 751, 129, 937, 863, 742, 694, 076, 438

第1趟  
 $dk=5$

256, 301, 694, 076, 438, 863, 742, 751, 129, 937

第2趟  
 $dk=3$

076, 301, 129, 256, 438, 694, 742, 751, 863, 937

第3趟  
 $dk=1$

076, 129, 256, 301, 438, 694, 742, 751, 863, 937

(取 $dk=5, 3, 1$ )

10.1 概述

10.2 插入排序

10.3 交换排序

10.4 选择排序

10.5 归并排序

10.6 基数排序

### 交换排序的基本思路

两两比较待排序记录的关键码，如果发生逆序（即排列顺序与排序后的次序正好相反），则交换之，直到所有记录都排好序为止。

交换排序的主要算法有：

- 1) 冒泡排序
- 2) 快速排序

### 1) 冒泡排序

**思路：**每趟不断将记录两两比较，并按“前小后大”（或“前大后小”）规则交换。

**优点：**每趟结束时挤出一个最大值到最后面位置，同时部分理顺其他元素；一旦下趟没有交换发生，提前结束排序。

**前提：**顺序存储结构

## 1) 冒泡排序

例：关键字序列  $T=(21, 25, 49, 25^*, 16, 08)$ ，请写出冒泡排序的具体实现过程。

|     |    |    |     |     |     |    |
|-----|----|----|-----|-----|-----|----|
| 初态: | 21 | 25 | 49  | 25* | 16  | 08 |
| 第1趟 | 21 | 25 | 25* | 16  | 08  | 49 |
| 第2趟 | 21 | 25 | 16  | 08  | 25* | 49 |
| 第3趟 | 21 | 16 | 08  | 25  | 25* | 49 |
| 第4趟 | 16 | 08 | 21  | 25  | 25* | 49 |
| 第5趟 | 08 | 16 | 21  | 25  | 25* | 49 |

### 冒泡排序的算法分析

**最好情况：**初始排列已经有序，只执行一趟起泡，做 $n-1$ 次关键码比较，不移动对象。

**最坏情形：**初始排列逆序，算法要执行 $n-1$ 趟起泡，第 $i$ 趟 ( $1 \leq i < n$ ) 做了 $n-i$ 次关键码比较，执行了 $n-i$ 次对象交换。此时的比较总次数 $KCN$ 和记录移动次数 $RMN$ 为：

$$KCN = \sum_{i=1}^{n-1} (n-i) = \frac{1}{2} n(n-1)$$

$$RMN = 3 \sum_{i=1}^{n-1} (n-i) = \frac{3}{2} n(n-1)$$

## 冒泡排序的算法分析

时间效率： $O(n^2)$ ，因为要考虑最坏情况

空间效率： $O(1)$ ，只在交换时用到一个缓冲单元

稳定性：稳定，25和25<sup>\*</sup>在排序前后的次序未改变



冒泡排序的优点：每一趟整理元素时，不仅可以完全确定一个元素的位置（挤出一个泡到表尾），还可以对前面的元素作一些整理，所以比一般的排序要快。

有没有比冒泡排序更快的算法？

快速排序法:每趟都能准确定位不止1个元素！

## 2) 快速排序

从待排序列中任取一个元素（例如取第一个）作为中心，所有比它小的元素一律前放，所有比它大的元素一律后放，形成左右两个子表；

然后再对各子表重新选择中心元素并依此规则调整，直到每个子表的元素只剩一个。此时便为有序序列了。

## 2) 快速排序

优点：因为每趟可以确定不止一个元素的位置，而且呈指数增加，所以特别快！

前提：顺序存储结构

例1: 关键字序列  $T=(21, 25, 49, 25^*, 16, 08)$ ，请写出快速排序的算法步骤。

设以首元素为枢轴中心

初态: 21, 25, 49, 25\*, 16, 08

第1趟: ( 16, 08 ), 21, ( 25, 25\*, 49 )

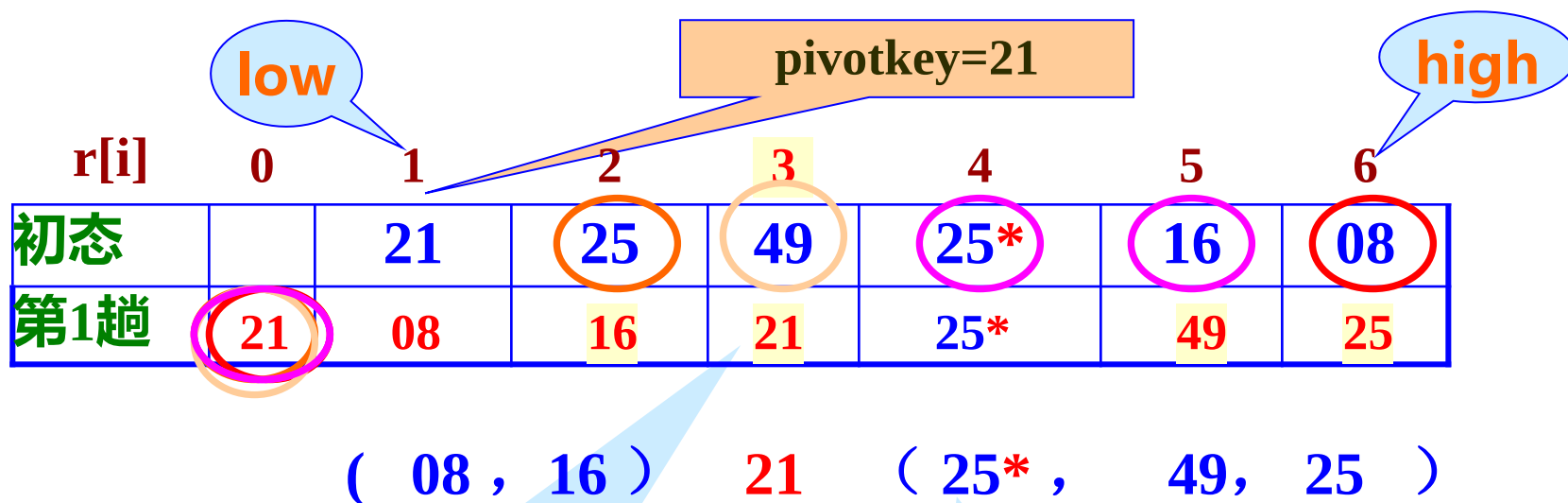
第2趟: (08), 16, 21, 25, (25\*, 49)

第3趟: 08, 16, 21, 25, 25\*, (49)

讨论:

1. 这种不断划分子表的过程，计算机如何自动实现？
2. “快速排序”是否真的比任何排序算法都快？

例2：关键字序列  $T=(21, 25, 49, 25^*, 16, 08)$ ，计算机如何实现快速排序算法的某一趟过程？交替/振荡式逼近



Low=high=3，本趟停止，将中枢纽点定位并返回位置信息

25\*跑到了前面，不稳定

例3：以关键字序列（256，301，751，129，937，863，742，694，076，438）为例，写出执行快速算法的各趟排序结束时，关键字序列的状态。

原始序列：256 301， 751， 129， 937， 863， 742， 694， 076， 438

第1趟 076， 129， 256， 751， 937， 863， 742， 694， 301， 438

第2趟 076， 129， 256， 438， 301， 694， 742， 751， 863， 937

第3趟 076， 129， 256， 301， 438 694， 742， 751， 863， 937

第4趟 076， 129， 256， 301， 438， 694， 742， 751， 863， 937

时间效率：  $O(n\log_2 n)$ ，因为每趟确定的元素呈指数增加

空间效率：  $O(\log_2 n)$ ，因为递归要用栈(存每层low， high 和pivot)

稳定性： 不稳定，因为有跳跃式交换。

**10.1 概述**

**10.2 插入排序**

**10.3 交换排序**

**10.4 选择排序**

**10.5 归并排序**

**10.6 基数排序**



### 选择排序基本思路

每一趟在后面 $n-i$ 个待排记录中选取关键字最小的记录作为有序序列中的第 $i$ 个记录。

### 选择排序实现算法

- 1) 简单选择排序
- 2) 锦标赛排序
- 3) 堆排序

### 1) 简单选择排序

每经过一趟比较就找出一个最小值，与待排序列最前面的位置互换即可。

首先，在 $n$ 个记录中选择最小者放到 $r[1]$ 位置；

然后，从剩余的 $n-1$ 个记录中选择最小者放到 $r[2]$ 位置；

...

如此进行下去，直到全部有序为止。

### 1) 简单选择排序

优点：实现简单

缺点：每趟只能确定一个元素，表长为 $n$ 时需要 $n-1$ 趟

前提：顺序存储结构

例：关键字序列 $T = (21, 25, 49, 25^*, 16, 08)$ ，请给出简单选择排序的具体实现过程。

最小值08与 $r[1]$ 交换位置

原始序列： 21, 25, 49, 25\*, 16, 08

第1趟 08, 25, 49, 25\*, 16, 21

第2趟 08, 16, 49, 25\*, 25, 21

第3趟 08, 16, 21, 25\*, 25, 49

第4趟 08, 16, 21, 25\*, 25, 49

第5趟 08, 16, 21, 25\*, 25, 49

时间效率：  $O(n^2)$ ，虽移动次数较少，但比较次数仍多。

空间效率：  $O(1)$ ，没有附加单元（仅用到1个temp）

算法的稳定性：不稳定，因为排序时，25\*到了25的前面。

简单选择排序的算法如下

```
void SelectSort(SqList &L ) { //对顺序表L作简单选择排序
    for (i=1; i<L.length; ++i){
        j = SelectMinKey(L,i); //在r[i...L.length]中选择最小记录并定位
        if( i!=j ) r[i] ↔ r[j]; //与第i个记录交换
    } //for
} //SelectSort
```

讨论：能否利用（或记忆）首趟的 $n-1$ 次比较所得信息，从而尽量减少后续比较次数呢？

锦标赛排序和堆排序

## 2) 锦标赛排序(又称树形选择排序)

基本思想:

与体育比赛时的淘汰赛类似。

首先对  $n$  个记录的关键字进行两两比较, 得到  $\lceil n/2 \rceil$  个**优胜者**(关键字小者), 作为第一步比较的结果保留下来。然后在这  $\lceil n/2 \rceil$  个较小者之间再进行两两比较, ..., 如此重复, 直到选出最小关键字的记录为止。



## 2) 锦标赛排序(又称树形选择排序)

优点：减少比较次数，加快排序速度

缺点：空间效率低

例：关键字序列 $T=(21, 25, 49, 25^*, 16, 08, 63)$ ，请给出锦标赛排序的具体实现过程。

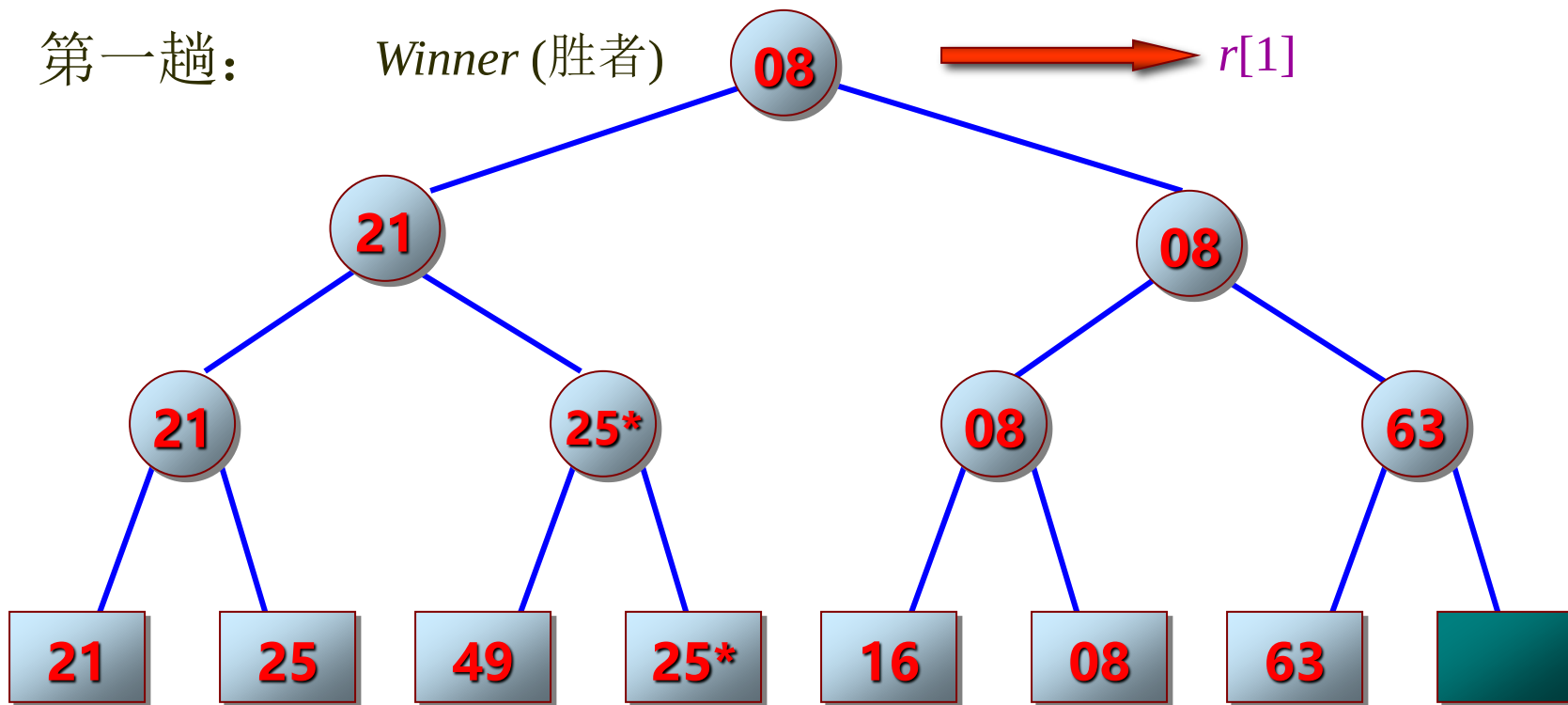
注：为便于自动处理，建议每个记录多开两个特殊分量：

| key | otherinfo | Index(结点位置编号) | Tag (是否参加比较) |
|-----|-----------|---------------|--------------|
|-----|-----------|---------------|--------------|

第一趟:

Winner (胜者)

08

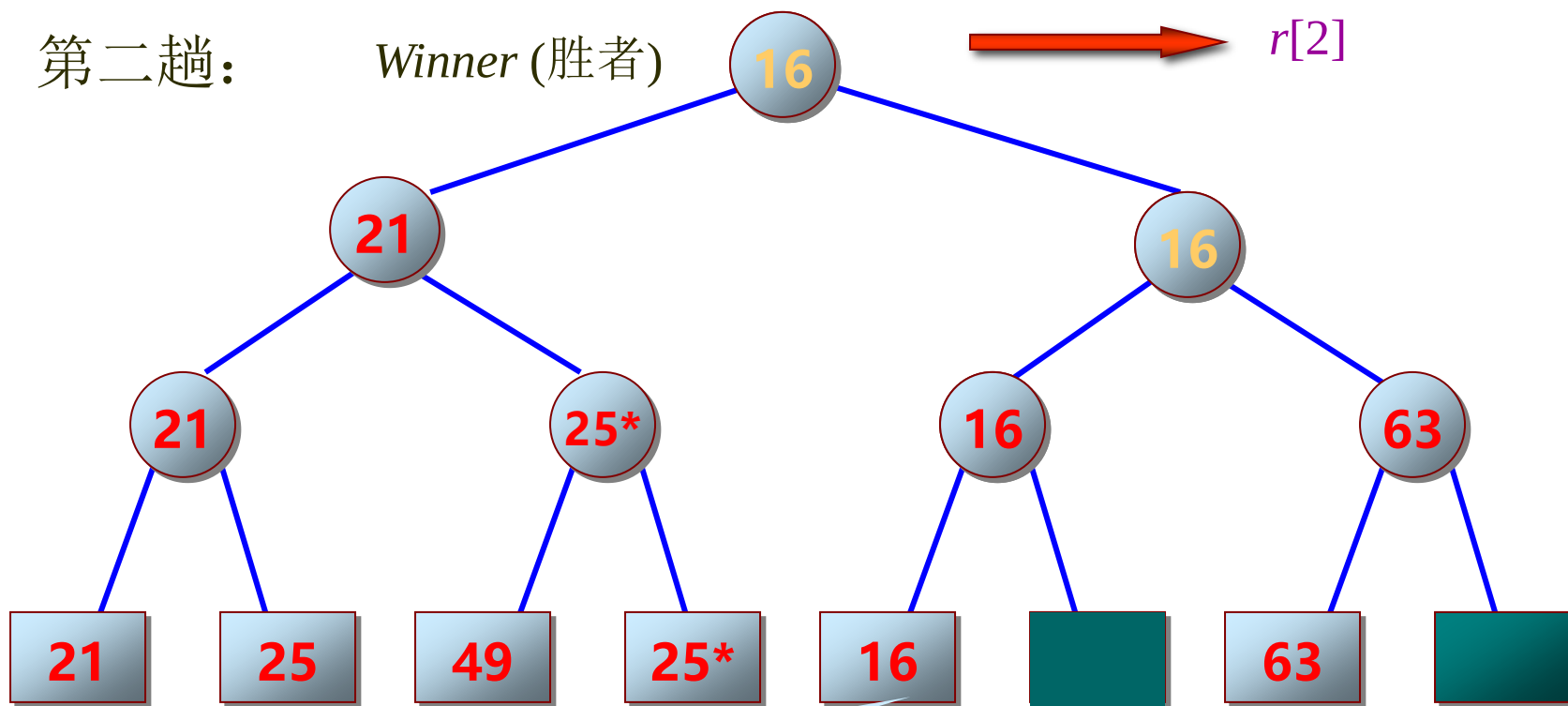
 $r[1]$ 

初态:

补足 $2^k$  ( $k=\lceil \log_2 n \rceil$ )个叶子结点

第二趟:

Winner (胜者)

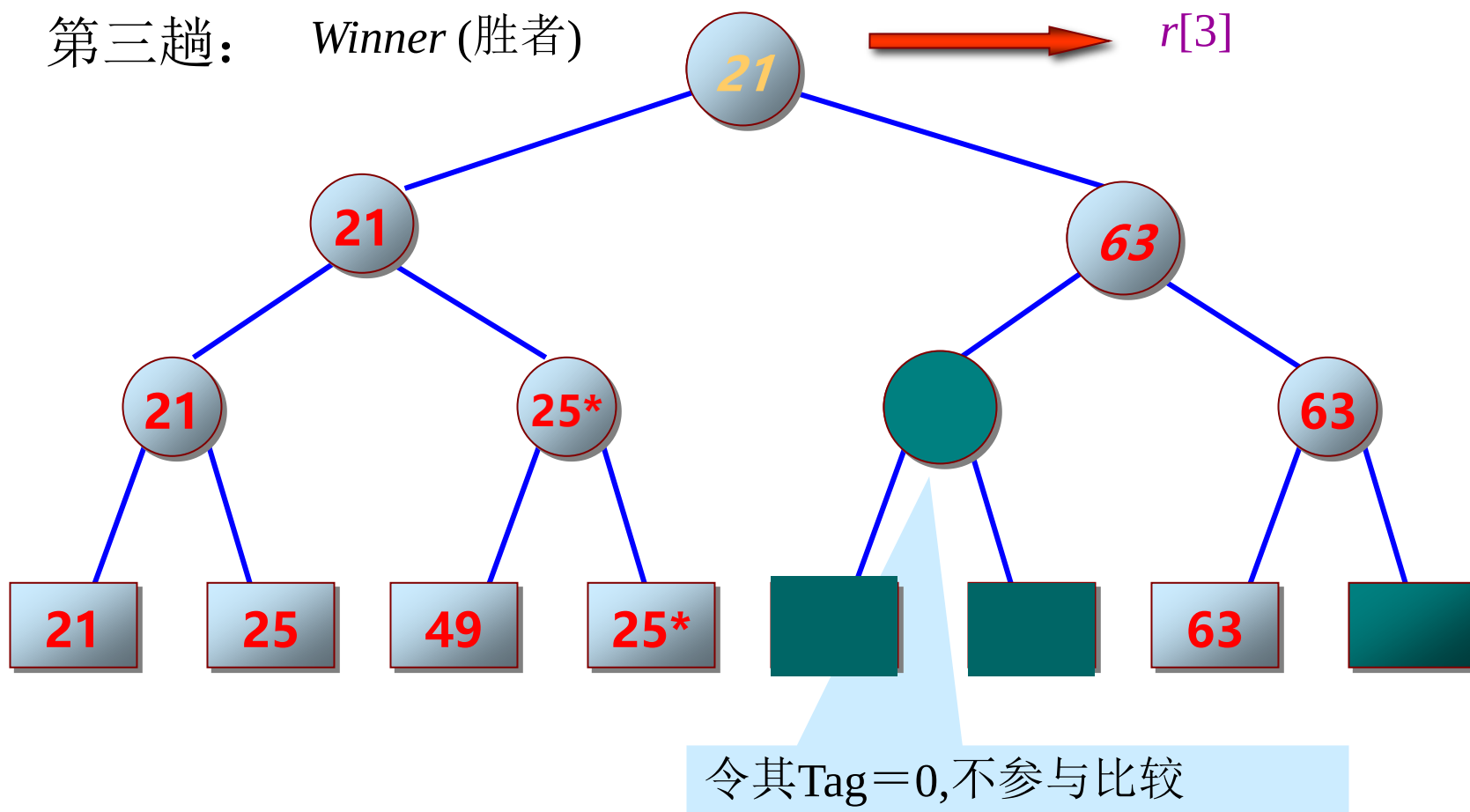


求次小值16时,只需比较2次,即只比较 $\lfloor \log_2 n \rfloor - 1$ 次

令其Tag=0,不参与比较

第三趟:

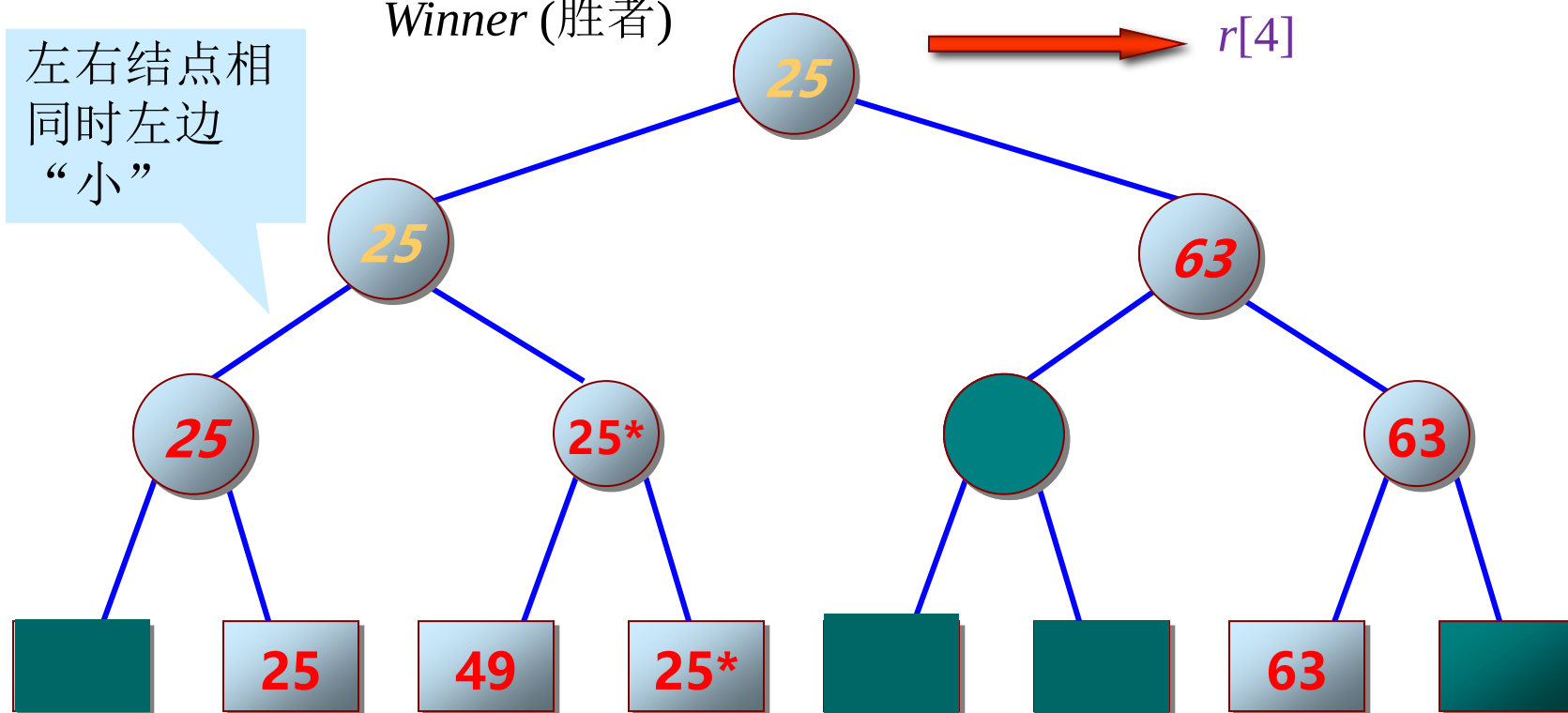
Winner (胜者)



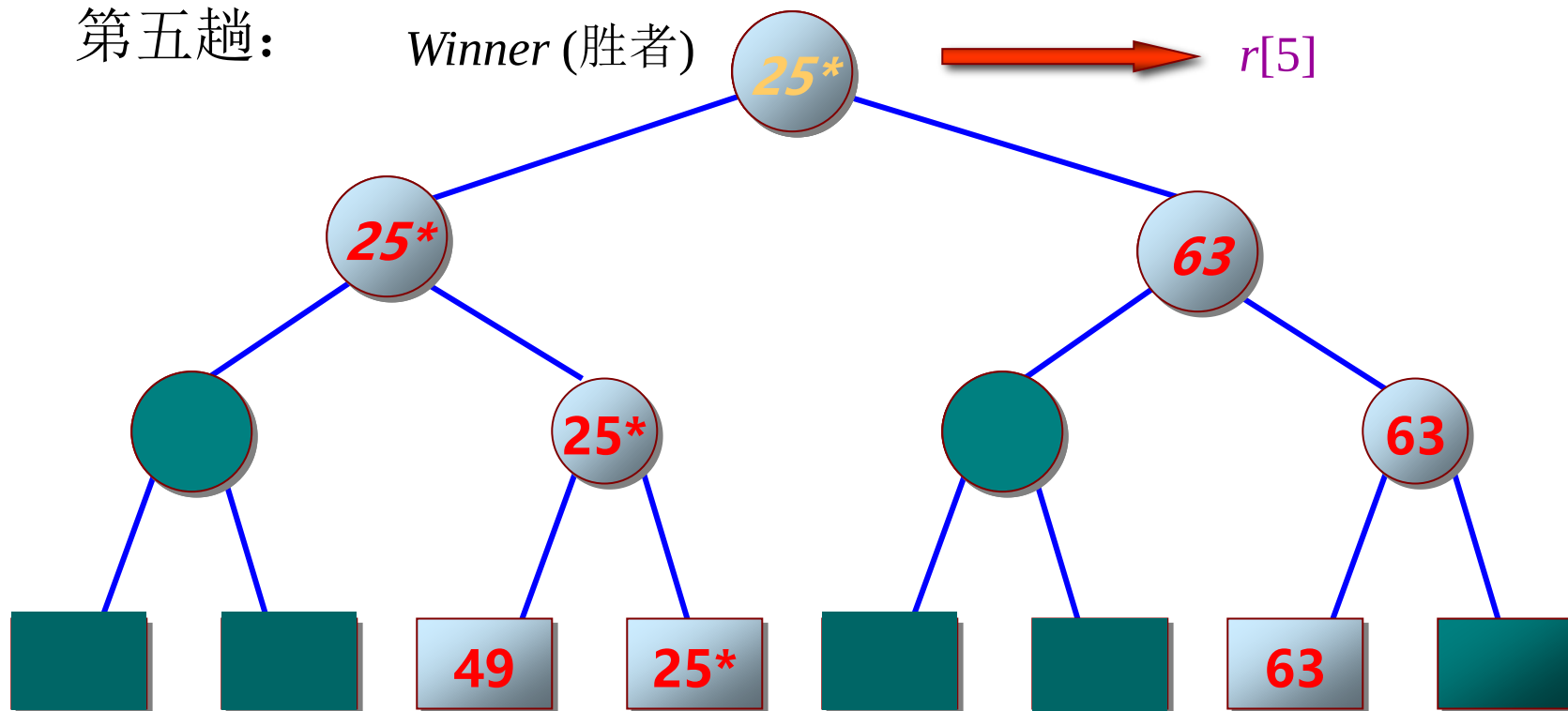
第四趟:

左右结点相  
同时左边  
“小”

Winner (胜者)

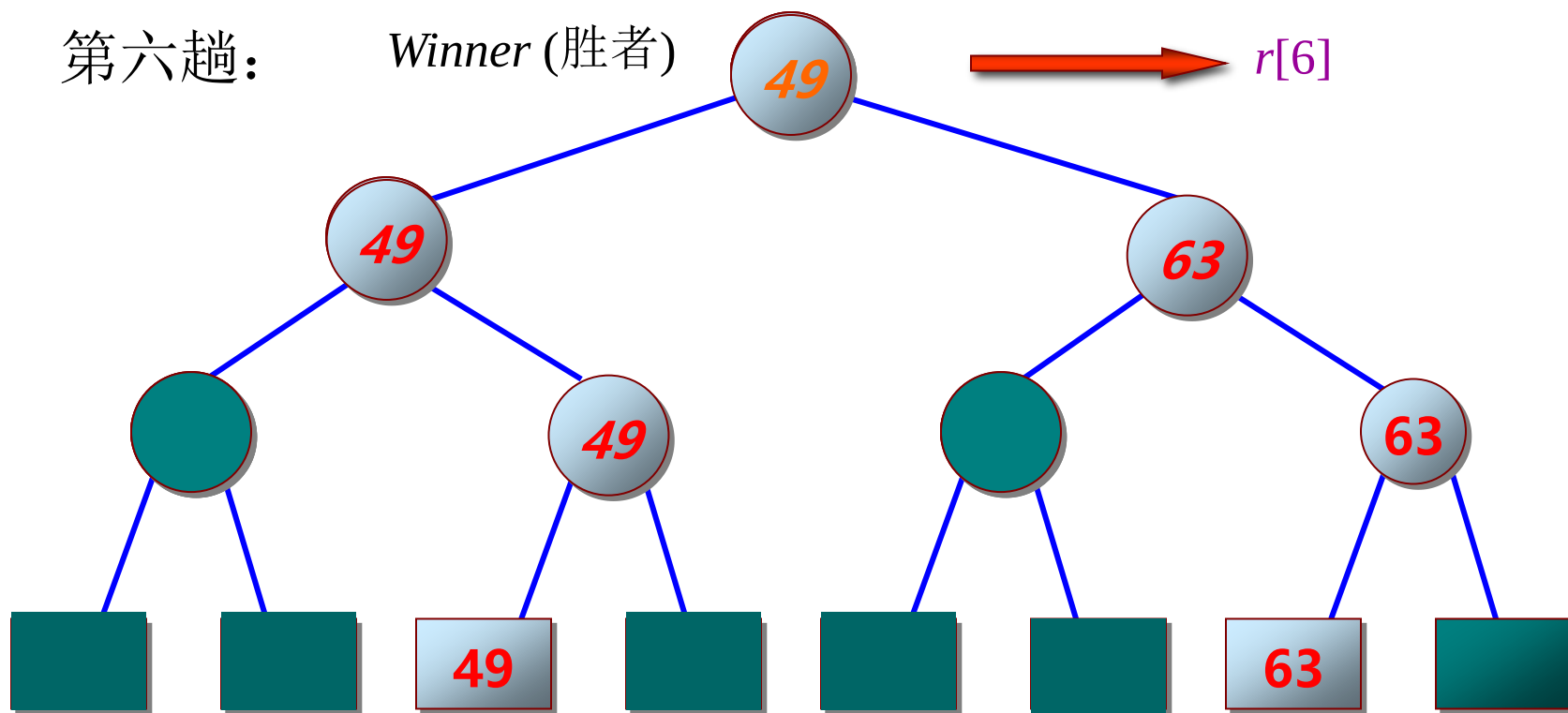


第五趟:



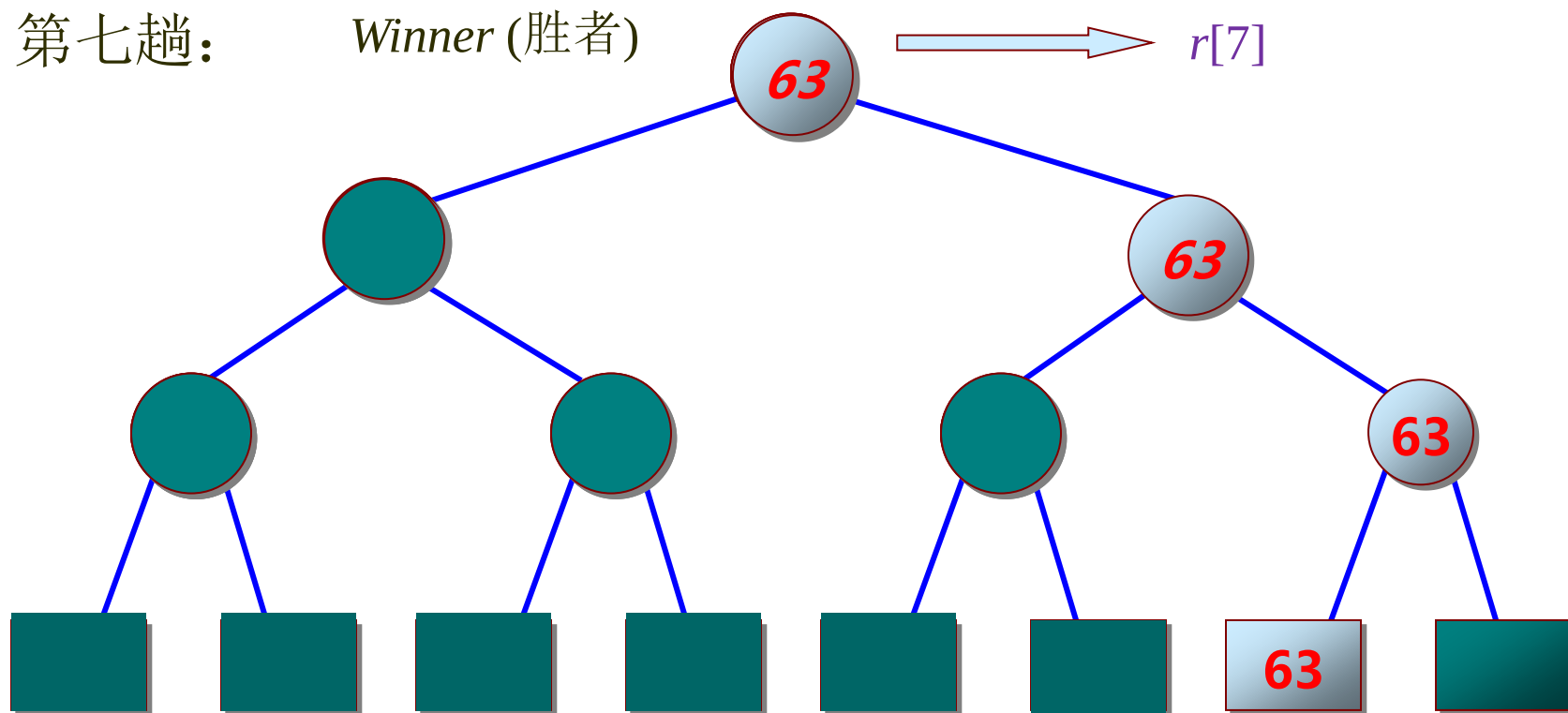
第六趟:

Winner (胜者)



第七趟:

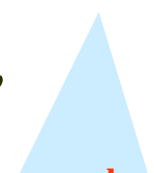
Winner (胜者)





算法分析：

- 锦标赛排序构成的树是**完全（满）二叉树**，其深度为  $\lfloor \log_2 n \rfloor + 1$ ，其中  $n$  为待排序元素个数。
- 时间复杂度：  $O(n \log_2 n)$ ，  $n$  个记录各自比较约  $\log_2 n$  次
- 空间效率：  $O(n)$ ， 胜者树的附加内结点共有  $n_0 - 1$  个！
- 稳定性： 稳定， 可事先约定左结点“小”


$$n_0 = 2^k = \text{叶子总数}$$

讨论：锦标赛排序巧妙利用了第一次和前若干次扫描的结果，加快了排序速度。还有没有同类方法？

堆排序

## 3) 堆排序

堆的定义：设有 $n$ 个元素的序列  $k_1, k_2, \dots, k_n$ ，当且仅当满足下述关系之一时，称之为堆。

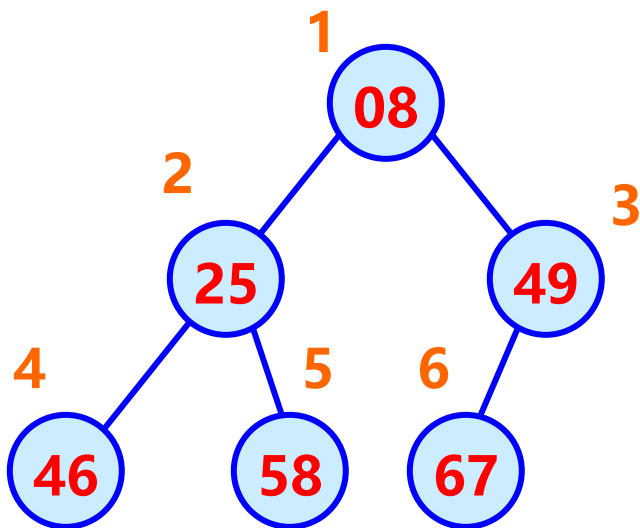
$$\left\{ \begin{array}{l} k_i \leq k_{2i} \\ k_i \leq k_{2i+1} \end{array} \right. \quad \text{或者} \quad \left\{ \begin{array}{l} k_i \geq k_{2i} \\ k_i \geq k_{2i+1} \end{array} \right. \quad i=1, 2, \dots, n/2$$

### 3) 堆排序

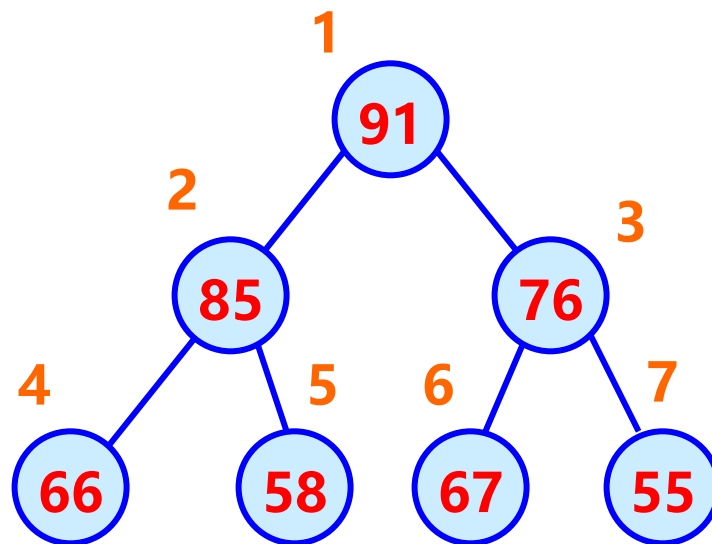
解释：如果让满足以上条件的元素序列 $(k_1, k_2, \dots, k_n)$ 依次排成一棵完全二叉树，则此树的特点是：

树中所有结点的值均大于（或小于）其左右孩子，此树的根结点（即堆顶）必最大（或最小）。

例：有序列 $T1=(08, 25, 49, 46, 58, 67)$ 和序列 $T2=(91, 85, 76, 66, 58, 67, 55)$ ，判断它们是否“堆”？



√  
(小根堆)  
(小顶堆)  
(最小堆)



√  
(大根堆)  
(大顶堆)  
(最大堆)

## 2. 怎样建堆？

步骤：从最后一个非终端结点开始往前逐步调整，让每个双亲大于（或小于）子女，直到根结点为止。

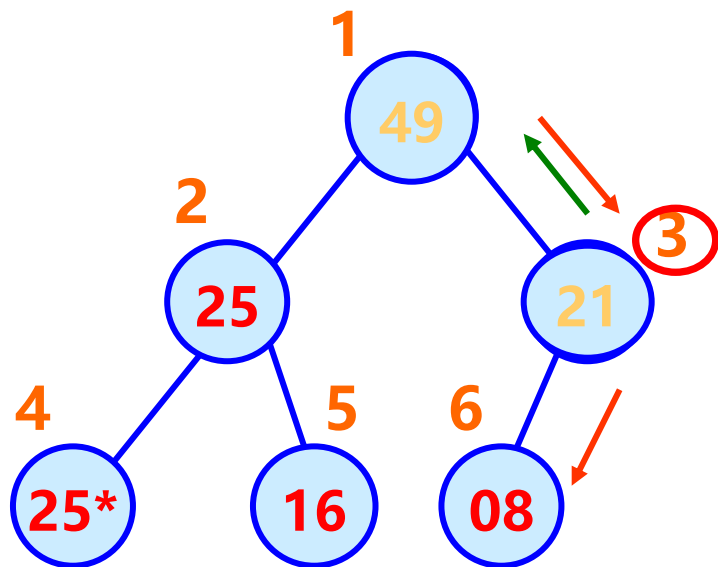
终端结点（即叶子）没有任何子女，无需单独调整

## 2. 怎样建堆？

例：关键字序列 $T = (21, 25, 49, 25^*, 16, 08)$ ，请建大根堆。

解：为便于理解，先将原始序列画成完全二叉树的形式：

这样可以很清晰地从 $\lfloor n/2 \rfloor$ 开始调整。



完全二叉树的第一个非终端结点  
编号必为 $\lfloor n/2 \rfloor$ ！！(性质5)

$i=3$ : 49大于08，不必调整；  
 $i=2$ : 25大于25\*和16，也不必调整；  
 $i=1$ : 21小于25和49，要调整！

而且21还应当向下比较！！

### 3. 怎样进行整个序列的堆排序？

难点：将堆的当前顶点输出后，如何将剩余序列重新调整为堆？

方法：将当前顶点与堆尾记录交换，然后仿建堆动作重新调整，如此反复直至排序结束。

即：将任务转化为—>

$H.r[i...m]$ 中除 $r[i]$ 外，其他都具有堆特征。

现调整 $r[i]$ 的值，使 $H.r[i...m]$ 为堆。



基于初始堆进行堆排序的算法步骤：

堆的第一个对象 $r[1]$ 具有最大的关键码，将 $r[1]$ 与 $r[n]$ 对调，把具有最大关键码的对象交换到最后；

再对前面的 $n-1$ 个对象，使用堆的调整算法，重新建立堆。结果具有次最大关键码的对象又上浮到堆顶，即 $r[1]$ 位置；

再对调 $r[1]$ 和 $r[n-1]$ ，然后对前 $n-2$ 个对象重新调整，...如此反复，最后得到全部排序好的对象序列。

10.1 概述

10.2 插入排序

10.3 交换排序

10.4 选择排序

10.5 归并排序

10.6 基数排序

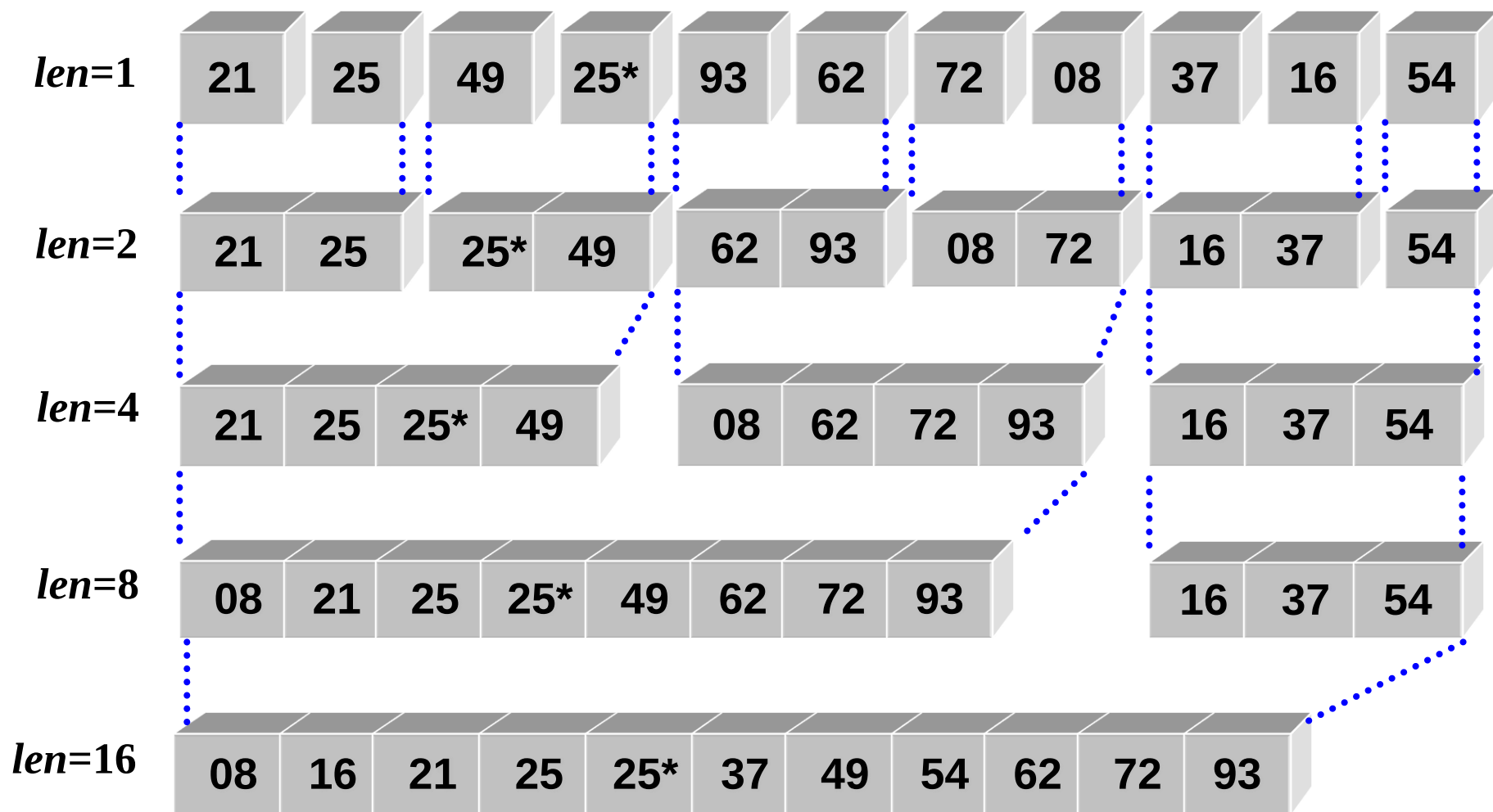
归并排序的基本思想是：将两个（或以上）的有序表组成新的有序表。

更实际的意义：可以把一个长度为 $n$ 的无序序列看成是 $n$ 个长度为1的有序子序列，首先做两两归并，得到 $\lceil n/2 \rceil$ 个长度为2的有序子序列；再做两两归并，...，如此重复，直到最后得到一个长度为 $n$ 的有序序列。

例：关键字序列 $T=(21, 25, 49, 25^*, 93, 62, 72, 08, 37, 16, 54)$ ，请给出归并排序的具体实现过程。

## 10.5

## 归并排序



整个归并排序仅需 $\lceil \log_2 n \rceil$ 趟，但每趟仍要比较和整理 $n$ 个元素

一趟归并排序算法：（两路有序并为一路）

```
void Merge (SR, &TR, i, m, n) {  
    // 将有序的SR[i...m]和SR[m+1...n]归并为有序的TR[i...n]  
    for(k=i, j=m+1; i<=m && j<=n; ++k) {  
        if ( SR[i]<= SR[j] ) TR[k]=SR[i++];  
        else TR[k]=SR[j++]; // 将两个SR记录由小到大并入TR  
    } // for  
    if (i<=m) TR[k...n]=SR[i...m]; // 将剩余的SR[i...m]复制到TR  
    if (j<=n) TR[k...n]=SR[j...n]; // 将剩余的SR[j...n]复制到TR  
} // Merge
```

这就是能提高排  
序速度的原因

递归形式的两路归并完整排序算法：

初次调用时为 (L, L, 1, length)

```
void MSort (SR[ ], &TR1[ ], s, t) {  
    // 将无序的SR[s...t]归并排序为TR1[s...t]  
  
    if ( s==t ) TR1[s]=SR[s];    // 当1 = length时返回  
    else {  
        m=(s+t)/2; // 将SR [s...t]平分为SR [s...m]和SR [m+1...t]  
        MSort (SR, &TR2, s, m); // 将SR 一分为二, 2分为4...  
        // 递归地将SR [s...m]归并为有序的TR2[s...m]  
        MSort (SR, &TR2, m+1, t);  
        // 递归地将SR [m+1...t]归并为有序的TR2[m+1...t]  
        Merge(TR2, TR1, s, m, t);  
        // 将TR2 [s...m]和TR2 [m+1...t]归并到TR1 [s...t]  
    }  
} // if  
} // MSort
```

简言之，先由“长”无序变成“短”有序，  
再从“短”有序归并为“长”有序。

归并排序算法分析：

时间效率：  $O(n\log_2 n)$

在每趟归并排序的操作中，要调用 $\lceil n/(2h) \rceil$ 次 $Merge()$ 算法，将 $SR[1\dots n]$ 中前后相邻且长度为 $h$ 的有序段进行两两归并，得到前后相邻、长度为 $2h$ 的有序段，并存放在 $TR[1\dots n]$ 中。另外，整个归并排序有 $\lceil \log_2 n \rceil$ “层”，所以算法总的时间复杂度为 $O(n\log_2 n)$ 。

归并排序算法分析：

空间效率：  $O(n)$

因为需要一个与原始序列同样大小的辅助序列（TR）。  
这正是此算法的缺点。

稳定性： 稳定



**10.1 概述**

**10.2 插入排序**

**10.3 交换排序**

**10.4 选择排序**

**10.5 归并排序**

**10.6 基数排序**

基数排序的基本思想是：

借助多关键字排序的思想对单逻辑关键字进行排序。即：  
用关键字不同的位值进行排序。

要讨论的问题：

1. 什么是“多关键字”排序？实现方法？
2. 单逻辑关键字怎样“按位值”排序？

例1：对一副扑克牌该如何排序？

若规定花色和面值的顺序关系为：

花色：♦ < ♣ < ♥ < ♠

面值：2 < 3 < 4 < 5 < 6 < 7 < 8 < 9 < 10 < J < Q < K < A

则可以先按花色排序，花色相同者再按面值排序；

也可以先按面值排序，面值相同者再按花色排序。

### 最高位优先法MSD

先设立4个花色“箱”，将全部牌按花色分别归入4个箱内（每个箱中有13张牌）；然后对每个箱中的牌按面值进行插入排序（或其它稳定算法）。

### 最低位优先法LSD

先按面值分成13堆（每堆4张牌），然后对每堆中的牌按花色进行排序（用插入排序等稳定的算法）

讨论：是借用MSD方式来排序好呢，还是借用LSD方式？

例：初始关键字序列 $T = (32, 13, 27, 32^*, 19, 33)$ ，请分别用MSD和LSD进行排序，并讨论其优缺点。

MSD：原始序列：32, 13, 27, 32\*, 19, 33

先按高位 $K_i^1$ 排序：(13, 19), 27, (32, 32\*, 33)

再按低位 $K_i^2$ 排序：13, 19, 27, 32, 32\*, 33

须将序列逐层分割成若干子序列，然后对各子序列分别进行排序。

讨论：是借用MSD方式来排序好呢，还是借用LSD方式？

例：初始关键字序列 $T = (32, 13, 27, 32^*, 19, 33)$ ，请分别用MSD和LSD进行排序，并讨论其优缺点。

LSD：原始序列： 32, 13, 27, 32\*, 19, 33

先按低位 $K_i^2$ 排序： 32, 32\*, 13, 33, 27, 19

再按高位 $K_i^1$ 排序： 13, 19, 27, 32, 32\*, 33

不必分成子序列，对每个关键字都是整个序列参加排序，每次排序时应选用稳定的排序方法

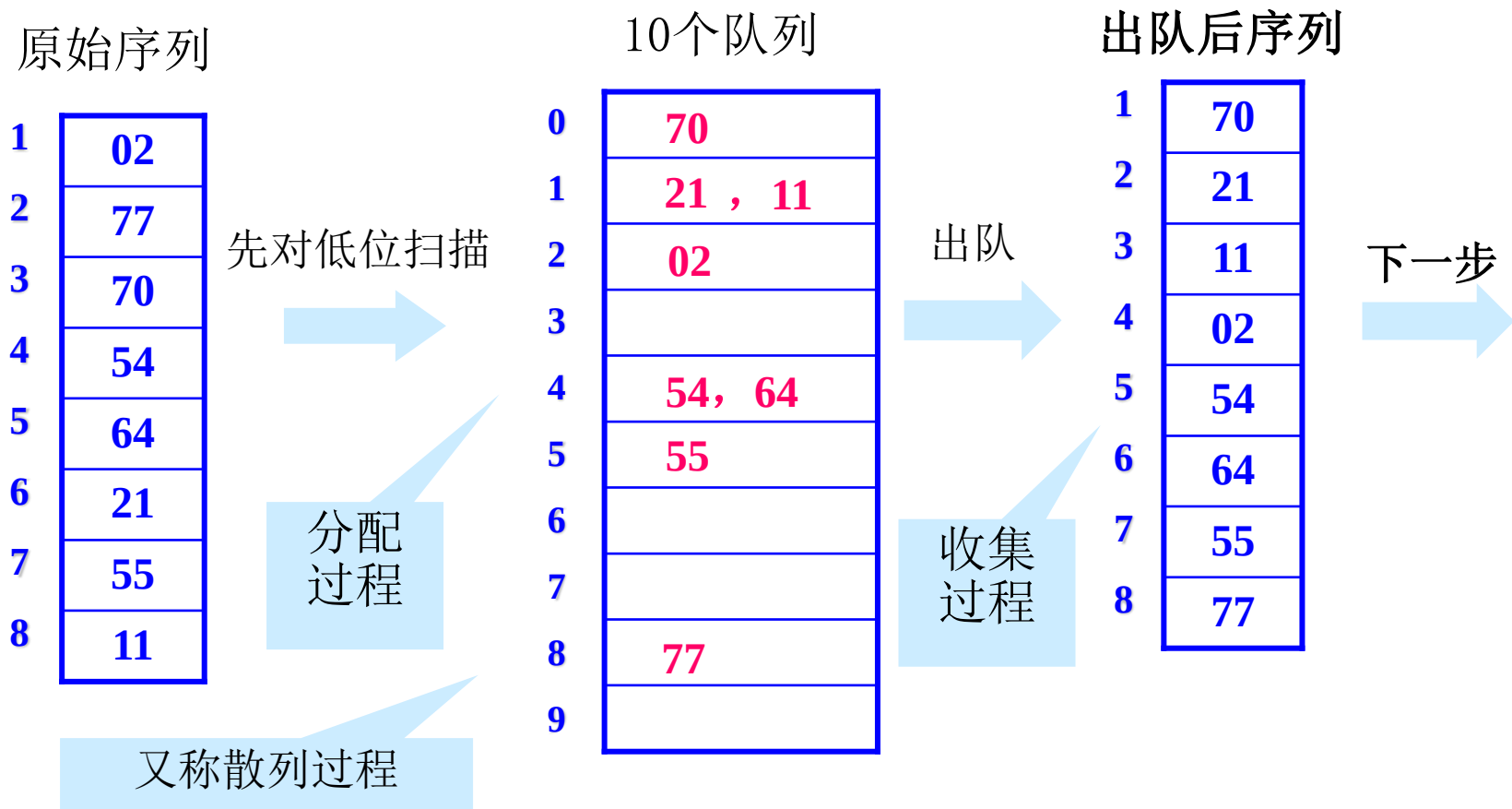
计算机怎样实现LSD算法？

例：T=（02， 77， 70， 54， 64， 21， 55， 11）， 用LSD排序。

分析：

①各关键字可视为2元组；②每位的取值范围是：0-9；即基数 $radix = 10$ 。因此，特设置10个队列，并编号为0-9。

例：T= (02, 77, 70, 54, 64, 21, 55, 11)，用LSD排序。





出队后序列

|   |    |
|---|----|
| 1 | 70 |
| 2 | 21 |
| 3 | 11 |
| 4 | 02 |
| 5 | 54 |
| 6 | 64 |
| 7 | 55 |
| 8 | 77 |

再对高位扫描

再次  
分配

再次入队

|   |         |
|---|---------|
| 0 | 02      |
| 1 | 11      |
| 2 | 21      |
| 3 |         |
| 4 |         |
| 5 | 54 , 55 |
| 6 | 64      |
| 7 | 70 , 77 |
| 8 |         |
| 9 |         |

再次出队

再次  
收集

再次出队后序列

|    |
|----|
| 02 |
| 11 |
| 21 |
| 54 |
| 55 |
| 64 |
| 70 |
| 77 |

小结：排序时经过了反复的“分配”和“收集”过程。当对关键字所有的位进行扫描排序后，整个序列便从无序变为有序了。

这种LSD排序方法称为： 基数排序

讨论：所用队列是顺序结构，浪费空间，能否改用链式结构？

用链队列来实现基数排序——链式基数排序

实现思路：

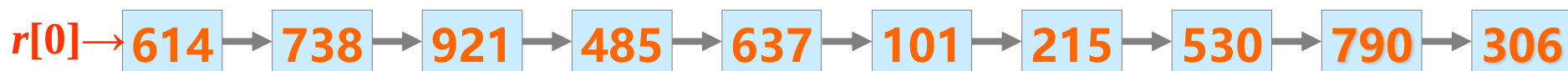
针对  $d$  元组中的每一位分量，把原始链表中的所有记录，按  $K_i^j$  的取值，让  $j = d, d-1, \dots, 1$

- ① 先“分配”到  $radix$  个链队列中去；
- ② 然后再按各链队列的顺序，依次把记录从链队列中“收集”起来；
- ③ 分别用这种“分配”、“收集”的运算逐趟进行排序；
- ④ 在最后一趟“分配”、“收集”完成后，所有记录就按其关键码的值从小到大排好序了。

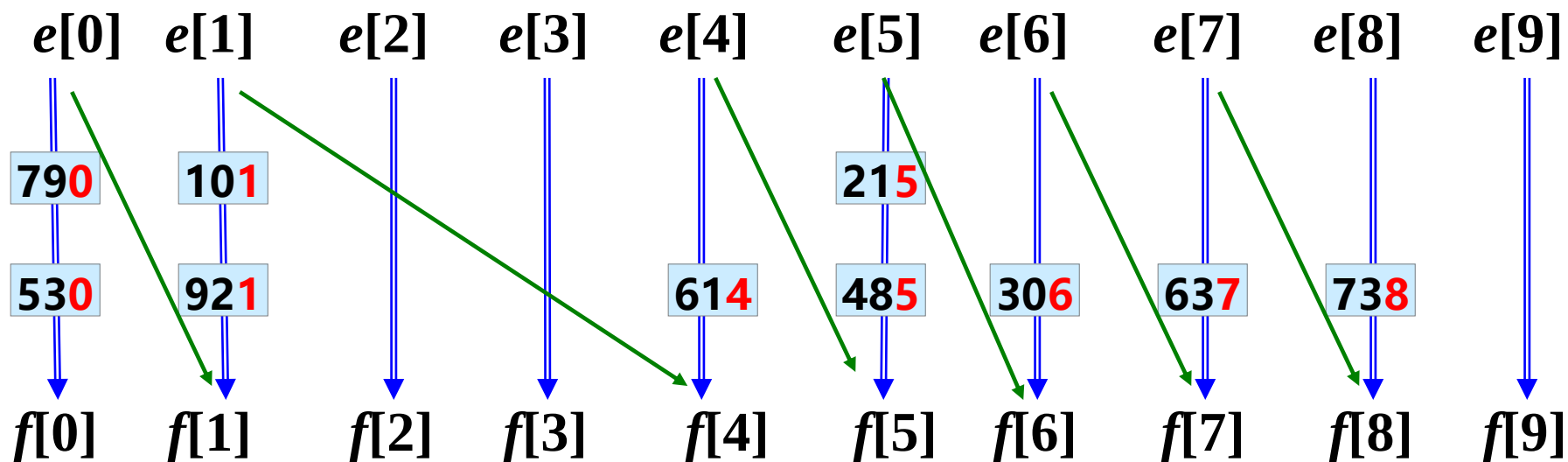
例:请实现以下关键字序列的链式基数排序:

$T = (614, 738, 921, 485, 637, 101, 215, 530, 790, 306)$

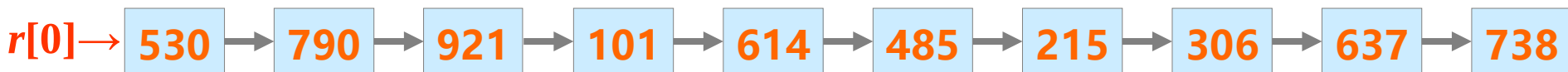
原始序列静态链表：



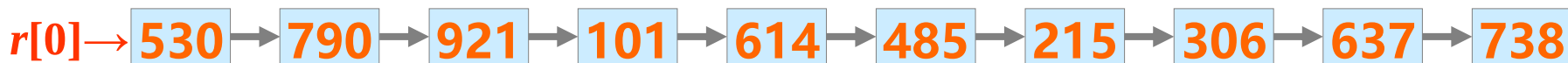
第一趟分配（从最低位  $i = 3$  开始排序,  $f[ ]$  是队首指针,  $e[ ]$  为队尾指针）



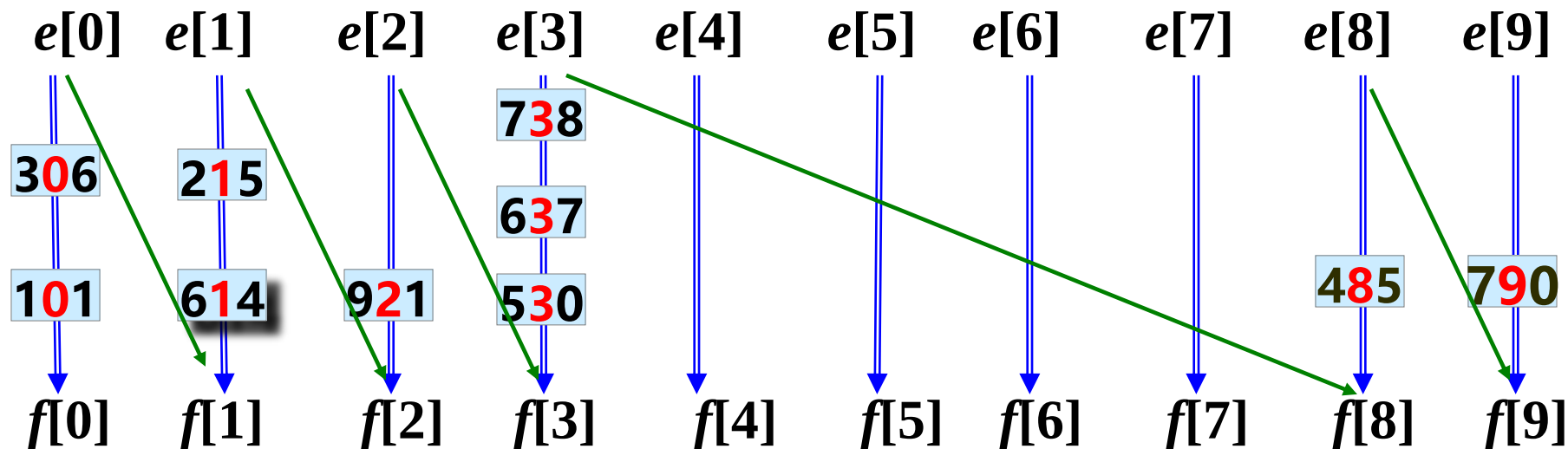
第一趟收集（让队尾指针  $e[i]$  链接到下一非空队首指针  $f[i+1]$  即可）



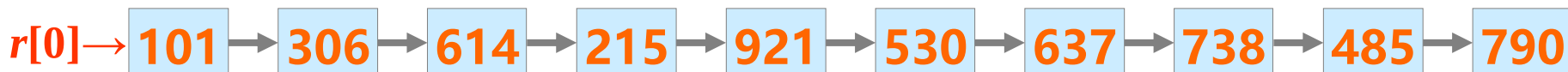
第一趟收集的结果：



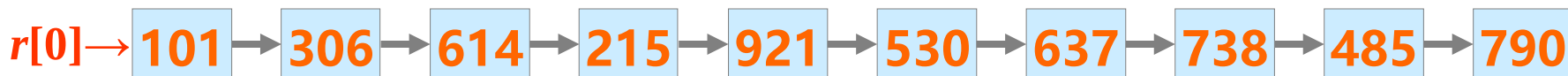
第二趟分配（按次低位  $i = 2$ ）



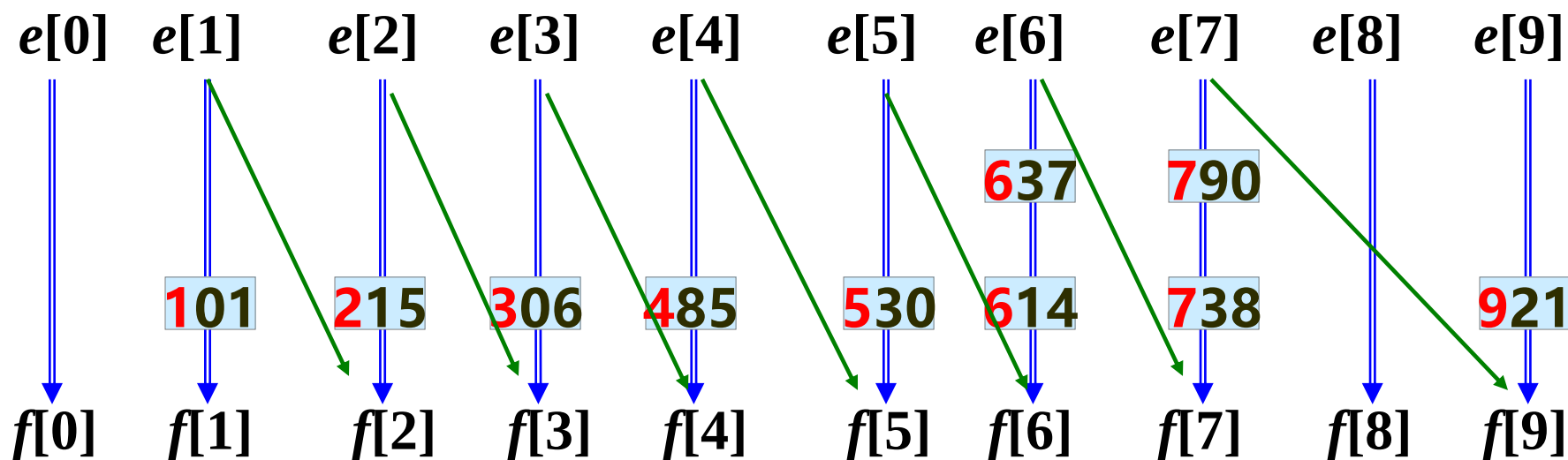
第二趟收集（让队尾指针  $e[i]$  链接到下一非空队首指针  $f[i+1]$ ）



第二趟收集的结果：



第三趟分配（按最高位  $i = 1$ ）



第三趟收集（让队尾指针  $e[i]$  链接到下一非空队首指针  $f[i+1]$ ）



排序的基本概念

插入排序

选择排序

交换排序

二路归并排序

基数排序



## 各种内部排序方法的比较

| 排序方法 | 最好情况         | 平均时间         | 最坏情况         | 辅助存储       | 稳定性 |
|------|--------------|--------------|--------------|------------|-----|
| 简单排序 | $O(n)$       | $O(n^2)$     | $O(n^2)$     | $O(1)$     | 稳定  |
| 快速排序 | $O(n \lg n)$ | $O(n \lg n)$ | $O(n^2)$     | $O(\lg n)$ | 不稳定 |
| 堆排序  | $O(n \lg n)$ | $O(n \lg n)$ | $O(n \lg n)$ | $O(1)$     | 不稳定 |
| 归并排序 | $O(n \lg n)$ | $O(n \lg n)$ | $O(n \lg n)$ | $O(n)$     | 稳定  |
| 基数排序 | $O(d(n+rd))$ | $O(d(n+rd))$ | $O(d(n+rd))$ | $O(rd)$    | 稳定  |
|      |              |              |              |            |     |
| 简单选择 | $O(n^2)$     | $O(n^2)$     | $O(n^2)$     | $O(1)$     | 不稳定 |
| 直接插入 | $O(n)$       | $O(n^2)$     | $O(n^2)$     | $O(1)$     | 稳定  |
| 折半插入 | $O(n \lg n)$ | $O(n \lg n)$ | $O(n \lg n)$ | $O(1)$     | 稳定  |
| 冒泡   | $O(n)$       | $O(n^2)$     | $O(n^2)$     | $O(1)$     | 稳定  |